Code review — colections-csharp

Ștefan, runde de review pe proiectul tău, cea mai nouă sus.

Runda 4 — 2026-10-27 — sha `65e9167`

Scop: fix-urile din runda 3, plus Pas 5 (`DictionarHash` , `MultimeHash` , `Pozitie`), Pas 6 (excepții și fail-fast) și Pas 7 (cât costă o căutare), commit-ul `65e9167` „code review + dictionar hash + multime hash + exceptii".

Metodă: build pe .NET 8, `Main` rulat întreg și comparat rând cu rând cu fișa. Apoi, pe o copie, încă 20 de teste mici: `Remove` pe tablou plin, `Insert` pe listă goală, `Remove` / `Clear` pe mulțime, `TryToTake` , `Add` pe cheie existentă, modificări în timpul unui `foreach` pe `Keys` , `Equals(null)` , câte hash-uri diferite are o tablă 8×8. Codul tău nu a fost atins.

Poarta de compilare: ✅ **0 erori**, ⚠️ **35 de warning-uri** (runda 3: 25).

Pe scurt:

- ✅ **Runda 3, pe fond:** compilează, `Main` cheamă toți pașii, iar Pas 1–4 ies **identic cu fișa, rând cu rând**. `Count` -ul dublu a dispărut, `ToArray` e scris o singură dată în bază, dublarea e în toate cele trei clase cu tablou, `Capacity => items.Length` , iar stiva se parcurge de la vârf. Cinci din cele șase 🔴 închise. 👏
- ✅ **Dicționarul merge:** găleata cu `& 0x7FFFFFFF` , redimensionarea la 7 chei (`Galeti=16` , exact ca în fișă), fiecare cheie mutată în găleata ei nouă. `Pozitie` cu `Equals` + `GetHashCode` dă `2 True True tura` . Pas 7: dicționarul e de **57.000 de ori** mai rapid.
- 🔴 Dar Pas 5a și Pas 6 nu ies ca în fișă: `TryToTake` șterge (B4), iar `Add` pe o cheie existentă suprascrie (B5).
- 🔴 `MultimeHash` își numără singură elementele și greșește (B3), iar `ListaTablou.Remove` crapă tot pe linia din runda 3 (B1).

Ieșirea, comparată cu fișa

Pas	Rând din fișă	La tine	De ce
1–4	toate	identice	✅
5a	`Ana, Elena, Ioana, Ion, Maria, Vlad`	`Ana, Elena, Ioana, Ion, Vlad`	B4
5a	`15, 17, 19, 20, 21, 30`	`15, 19, 20, 21, 30`	B4
5b, 5c	toate	identice	✅
6	`d.Adauga("Ana", 18) -> ArgumentException`	`... -> ok`	B5
6	`... may not execute.` (de 2 ori)	fără punctul de la final	M3
7	sute de ms / sutimi de ms	1595 ms / 0,028 ms	✅

🔴 Critice

B1 — `ListaTablou.cs:31` : `Remove` golește tot locul de la `Count`

Dovada: `[a, b, c, d]` → `Remove("a")` → `IndexOutOfRangeException` la `:31`. La fel pe orice listă plină (8 elemente, 16 elemente…).

Mecanism: e B4 din runda 3. L-ai reparat în `RemoveAt` (`:76`, acum `Count - 1`), dar `Remove` face **aceeași** mutare și golire, scrisă a doua oară, iar acolo a rămas `items[Count]`. După mutare, ultimul element ocupat e la `Count - 1`; `items[Count]` e primul loc liber și, pe tablou plin, nu există. Pas 2 nu-l prinde fiindcă `Remove("c")` se face pe 4 elemente într-un tablou de 8.

💡 Ai deja `IndexOf` și `RemoveAt`, amândouă corecte. Cum ar arăta `Remove` scris din ele, în 3 rânduri? Câte locuri ar mai rămâne unde poți greși indicele?

B2 — `ListaInlantuita.cs:83-91` : `Insert` pe lista goală ignoră indicele

Dovada: pe lista goală, `Insert(5, "x")` → `[x] Count=1`, `Insert(-3, "x")` → `[x]`. Fișa, Pas 6: `ArgumentOutOfRangeException`. E M4 din runda 3, rămas neatins.

Mecanism: verificările se fac **în ordine**. Cazul special `Head == null` e la `:83`, înaintea gărzii de la `:91`, deci pe lista goală garda nu e citită niciodată: orice indice ajunge să adauge. O gardă își face treaba doar dacă e **primul** rând din metodă; după ea, cazurile speciale lucrează numai cu indici deja corecți. În `RemoveAt` (`:128`) ai pus-o exact așa.

💡 Pe lista goală, care e singurul `index` valid la `Insert`? Dacă garda e prima, mai e nevoie de ramura `Head == null`, sau o acoperă `index == 0`?

B3 — `MultimeHash.cs:15`, `:21`, `:25` : `Count` ținut de mână lângă dicționar

Dovada:

Ce faci	`Count`	Afișarea
`{1, 2}` → `Remove(99)`	1	`IndexOutOfRangeException` la `ColectieAbstracta.cs:34`
`{1, 2}` → `Clear()`	2	`[0, 0]`
`Clear()` → `Add(7)`	3	`7, 0, 0`

Același `Count` are și `set` public în bază (`ColectieAbstracta.cs:9`), rămas din runda 3: `lista.Count = 10` → `IndexOutOfRangeException` la `ListaTablou.cs:122`; pe `ListaInlantuita`, `[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]`.

Mecanism: mulțimea are două numere care spun același lucru, `Count` al ei și `dictionar.Count`. `Remove` scade `Count` **înainte** să afle dacă a șters ceva (`:21`), iar `Clear` golește dicționarul și uită de `Count` (`:25`). De ce crapă afișarea: `ToArray` face tabloul de mărimea `Count` (1), iar `foreach` îi dă tot ce e în dicționar (2). E B5 din runda 2 (`Capacity` scris de mână lângă tablou), mutat în altă clasă: un număr care poate fi calculat nu se ține separat, fiindcă la prima metodă care uită să-l actualizeze începe să mintă.

💡 Fișa, Pas 3: `Count` e `abstract` în bază. Ce-ar scrie atunci `MultimeHash` pe un singur rând? Și ce face `ListaTablou` cu un `Count` pe care `Program` nu-l mai poate seta?

B4 — `DictionarHash.cs:105-106` : `TryToTake` șterge cheia

Dovada: `Maria` dispare din dicționar după `TryToTake("Maria", ...)`, deci lipsește din ambele rânduri sortate de la Pas 5a (vezi tabelul de sus).

Mecanism: „Try" din .NET (`TryGetValue`, `int.TryParse`) înseamnă **„fără excepție"**, nu „ia și scoate". Metoda face exact ce face `this[k]`, citește, dar în loc să arunce când cheia lipsește, întoarce `false`, iar valoarea iese prin `out`. O citire nu schimbă niciodată colecția. Aici ți-a jucat feste traducerea: `IncearcaSaIa` (încearcă să iei valoarea) a devenit `TryToTake`, iar „take" se citește ca „scoate". E un argument în plus la M1, numele din contract.

💡 Uită-te la `ContainsKey` (`:79`) și la `this[k]` (`:24–27`). Ce are `TryToTake` în plus față de ele? Ce rânduri rămân dacă scoți asta?

B5 — `DictionarHash.cs:32`, `:56–61`: `Add` pe o cheie existentă suprascrie

Dovada: `d.Add("Ana", 17); d.Add("Ana", 18)` → fără excepție, `d["Ana"] = 18`. Fișa, Pas 6: `ArgumentException`, „An item with the same key has already been added. Key: Ana".

Mecanism: dicționarul are două uși de scriere cu **contracte diferite**. `d[k] = v` înseamnă „pune, oricum": înlocuiește sau adaugă. `Add(k, v)` înseamnă „cheia asta e nouă", iar dacă nu e, e o greșeală a celui care cheamă. De-aia `Add` aruncă: dacă ar suprascrie în tăcere, o valoare veche s-ar pierde fără ca nimeni să afle. La tine indexerul cheamă `Add` (`:32`), deci au același comportament, iar rândul comentat de la `:31` arată că ai simțit asta, doar că ai pus excepția pe ușa greșită.

💡 Cele două uși au în comun găsirea perechii și adăugarea în găleată. Ce diferă între ele, la cheie existentă? O metodă `private` cu parametrul `bool` ar acoperi ambele.

B6 — `DictionarHash.cs:114`, `:132`: fail-fast pe `Count`, nu pe o versiune

Dovada: `{0, 1, 2, 3, 4}`, `foreach` pe `Keys`, la cheia 3 faci `Remove(0)` + `Add(6)` → **fără excepție**, `foreach` vede `0,1,2,3,4,6`, deci și o cheie ștearsă. Iar `d["a"] = 100` în timpul unui `foreach` pe `Values` trece la fel.

Mecanism: `Count` măsoară **mărimea**, nu **schimbarea**. Un `Remove` și un `Add` se anulează, iar o suprascriere nu schimbă mărimea deloc, deci `versiune != Count` nu vede nimic. De ce aruncă totuși uneori? Când modificarea e în **aceeași găleată** pe care o parcurgi, aruncă `ListaInlantuita` din interior, cu `versiune`-a ei. Te salvează din noroc, nu din dicționar. Fișa, Pas 6: „`DictionarHash` are propriul contor", crescut la orice modificare reușită, exact ca `versiune` din `ColectieAbstracta`.

💡 Ai deja tiparul în `ListaTablou.GetEnumerator` (`:117–126`). Unde trebuie crescut contorul în dicționar, ca să-l prindă și pe `d["a"] = 100`?

🟡 Importante

- **M1** — numele din contract, tot de decis la ședință: `Add` / `Remove` / `TryToTake` în loc de `Adauga` / `Sterge` / `IncearcaSaIa`. B4 e prima greșeală care vine direct din traducere, iar etichetele din `Pas6` (`"lista.Insereaza(5, 6)"`) nu mai descriu ce cheamă codul.
- **M2** — Template Method pe jumătate: `ToArray` e acum scris cu `foreach (T x in this)` ✅, dar `Contains` din bază (`ColectieAbstracta.cs:18`) parcurge `ToArray()`, adică copiază toată colecția la fiecare căutare: un `Contains` pe 100.000 de elemente alocă **390 KB**. Iar `MultimeHash.Contains` (`:63`) **ascunde** `Contains` din bază, nu-l suprascrie (`CS0108`, același warning ca la B2 din runda 3): prin `ColectieAbstracta<int>`, 200 de `Contains` durează **2164 ms**, prin `IColectie<int>` 0,0 ms. Fișa: „`Contine` e suprascris", deci `virtual` în bază, `override` în mulțime.
- **M3** — mesajele din Pas 6, a cărui „Gata când" cere `ex.Message` identic cu tabelul: `KeyNotFoundException` are „Cheia 'Ion' nu exista." (`DictionarHash.cs:26`), iar „Collection was modified..." n-are punct la final, în 6 locuri (`ListaTablou.cs:124`, `ListaInlantuita.cs:241`, `StivaTablou.cs:89`, `CoadaCirculara.cs:78`, `DictionarHash.cs:122`, `:140`).
- **M4** — `Pozitie` (`:14–25`): `Equals(null)` și `Equals("x")` → `NullReferenceException` la `:17`, fiindcă `as` întoarce `null` pe alt tip. `Equals` nu aruncă niciodată, întoarce `false`. Iar `Rand + Coloana` dă pe o tablă 8×8 doar **15 hash-uri diferite pentru 64 de poziții**: (1,5), (2,4), (5,1) sunt toate 6, deci ajung în aceeași găleată. `HashCode.Combine(Rand, Coloana)`.
- **M5** — warning-urile au crescut de la 25 la 35. 12 vin dintr-un singur `?`: `buckets` e declarat `ListaInlantuita<...>[]?` (`DictionarHash.cs:9`), deși nu e niciodată `null`. `Node<T>` și `PairNode<K, V>` sunt tot `public`, cu câmpuri publice.

🟢 Cleanups

- **C1** — cod mort în dicționar: câmpul `node` (`DictionarHash.cs:8`), constructorul `PairNode()` făcut doar pentru el și câmpul `next` (`PairNode.cs:7`), nefolosit fiindcă găleata e o `ListaInlantuita`.
- **C2** — `Reuniune` (`MultimeHash.cs:39`): `if (!Contains(item))` verifică mulțimea **asta**, nu rezultatul, și oricum `result.Add` nu pune dubluri. Rândul poate dispărea.
- **C3** — `ExYield` și `//IColectie`, `//ILista`, `//ICoada` în cod; `RASPUNSURI.txt` lipsește în continuare, iar acum îl cer și Pas 5, Pas 7 și Provocarea.

Before / After

	Acum	Corect
B1	`items[Count] = default;`	`items[Count - 1] = default;` sau `RemoveAt(IndexOf(item))`
B2	`if (Head == null) { ... }` apoi garda	garda `index > Count || index < 0` pe primul rând
B3	`Count++` / `Count--` în `MultimeHash`	`public override int Count => dictionar.Count;` (`abstract` în bază)
B4	`buckets[HashKey(key)].Remove(pair); Count--;`	doar `value = pair.Value; return true;`
B5	`Add` suprascrie, `this[k] = v` cheamă `Add`	`Add` → `ArgumentException` pe cheie existentă; `this[k] = v` înlocuiește sau adaugă
B6	`int versiune = Count;`	contor propriu, `versiune++` la `Add`, `Remove`, `Clear`, `this[k] = v`

Q&A

1. `TryToTake` ar trebui să întoarcă aceeași valoare ca `this[k]`. Ce le deosebește atunci, și de ce .NET le are pe amândouă?
2. În B6, de ce aruncă uneori `foreach` pe `Keys`, deși dicționarul tău nu observă nimic? Care clasă aruncă de fapt, și cu ce contor?
3. `MultimeHash.Contains` dă 0,0 ms prin `IColectie<int>` și 2164 ms prin `ColectieAbstracta<int>`. Pe același obiect, cine alege care metodă se cheamă, și când?

Răspunsurile (și cele 9 din rundele 1–3) în `RASPUNSURI.txt`, în repo.

Următorul pas: B1–B6, până când Pas 5a și Pas 6 ies identic cu fișa, apoi `RASPUNSURI.txt` și Provocarea (Academy cu `DictionarHash<Guid, T>`).

Runda 3 — 2026-10-20 — sha `2db692a`

Fix-urile din runda 2 + Pas 3 (`ColectieAbstracta`), Pas 4 (stiva, coada), început de Pas 5.

- ✅ **B1** nu compila: închis, `Main` cheamă toți pașii
- ✅ **B2** două `Count` -uri: închis · 🔴 `set` public rămas (→ runda 4, B3)
- ✅ **B3** `ToArray` întorcea tabloul din interior: închis (scris o dată, în bază)
- ✅ **B4** golirea locului: închis în `RemoveAt` , garda `>=` reparată · 🔴 repetat în `Remove` (→ runda 4, B1)
- ✅ **B5** tabloul creștea cu 1: închis în toate trei clasele, `Capacity => items.Length`
- ✅ **B6** stiva parcursă de la bază: închis
- ⏸ **M1** numele din contract: de decis la ședință (→ runda 4, M1)
- ⚠️ **M2** Template Method: `ToArray` din `foreach` ✅, `GetEnumerator` ne-generic scos ✅ · `Contains` tot din `ToArray` (→ runda 4, M2)
- 🔴 **M3** nod public + warning-uri: 25 → 35 (→ runda 4, M5)
- ⚠️ **M4** index greșit: `ListaTablou` închis · 🔴 `ListaInlantuita.Insert` pe lista goală (→ runda 4, B2)
- ⚠️ **M5** nodul dicționarului: `PairNode` cu cheie + valoare ✅ · câmpul `node` rămas (→ runda 4, C1)
- ✅ **C1** `Tail = new Node<T>()` : închis · ✅ **C2** `", "` : închis · ⚠️ **C3** `CoadaCirculara` cu `private` ✅, notițe în cod + `RASPUNSURI.txt` lipsă (→ runda 4, C3)

Runda 2 — 2026-10-17 — sha `a8e794b`

Fix-urile din runda 1 + Pas 2, `ListaTablou<T>` .

- ✅ **B1** `ToString` dubla primul element: închis (mutat în `ColectieAbstracta`)
- ✅ **B2** `AsVector` cu `Count + 1` : închis în `ListaInlantuita` · 🔴 repetat în `ListaTablou.ToArray` (→ runda 3, B3)
- ✅ **B3** `Tail = Head` în `Remove` : închis
- ✅ **B4** `GetEnumerator` ne-generic în `ListaTablou` : închis
- 🔴 **B5** `Capacity` câmp separat de tablou: neînchis, mai grav (→ runda 3, B5)
- ✅ **B6** `Equals` pe `null` : închis, în toate cele 4 clase
- ⏸ **M1** numele din contract: de decis la ședință (→ runda 3, M1)

- 🔴 **M2** nod public + warning-uri: 14 → 25 (→ runda 3, M3)
- 🔴 **M3** index greșit tăcut: neatins (→ runda 3, M4)
- ✅ **M4** dublare scrisă de 2 ori, bucle de căutare: închis (`ResizeIfFull`)
- ⚠️ **C1** `Node()` + `Tail` gol în constructor: rămas · ⚠️ **C2** golirea locului: scrisă, dar pe indicele greșit (→ runda 3, B4) · 🔴 **C3** `RASPUNSURI.txt` lipsă

Runda 1 — 2026-10-16 — sha `ac84a76`

Pasul 1: `IColectie<T>`, `ILista<T>`, `ListaInlantuita<T>`.

- ✅ **B1** `foreach` / `GetEnumerator`: închis · ⚠️ `ToString` scris, dar dublează primul element (→ runda 2, B1)
- ✅ **B2** `RemoveAt(0)` scădea `Count` de două ori: închis · ⚠️ ascunderea lui prin `Count + 1` în `AsVector` a rămas (→ runda 2, B2)
- ✅ **B3** `Tail` după constructor, `Clear`, `Insert(0)`: închis · 🔴 o linie nouă greșită în `Remove` (→ runda 2, B3)
- ✅ **B4** listă goală și listă cu un element: închis
- ✅ **M1** `Head` folosit drept cursor: închis
- ⏸ **M2** numele din contract: de decis la ședință (→ runda 2, M1)
- 🔴 **M3** nod public + warning-uri: neatins, 13 → 14 (→ runda 2, M2)
- 🔴 **M4** index greșit tăcut: neatins (→ runda 2, M3)
- ✅ **C1** `using`-uri nefolosite · ⚠️ **C2** `Node()` rămas · 🔴 **C3** notițe în cod, `RASPUNSURI.txt` lipsă